

10 DSA Coding Questions — Explained Python Solutions

Exam/Interview Revision Guide

Based on the uploaded question paper. Each solution includes the core idea, Python code, and complexity.

1. Product of Array Except Self

Explanation: Idea: Build the answer using two passes. In the first pass, store the product of all elements to the left of each index. In the second pass, multiply by the product of all elements to the right. This avoids division and runs in $O(n)$ time.

Python Solution:

```
def productExceptSelf(nums):
    n = len(nums)
    ans = [1] * n

    left = 1
    for i in range(n):
        ans[i] = left
        left *= nums[i]

    right = 1
    for i in range(n - 1, -1, -1):
        ans[i] *= right
        right *= nums[i]

    return ans

# Example: [1,2,3,4] -> [24,12,8,6]
```

Complexity: Time: $O(n)$ | Extra space: $O(1)$ excluding the output array

2. Longest Consecutive Sequence

Explanation: Idea: Put every number into a set for $O(1)$ average membership checks. A number starts a sequence only when $num - 1$ is not present. Then count forward until the sequence ends.

Python Solution:

```
def longestConsecutive(nums):
    s = set(nums)
    longest = 0

    for num in s:
        if num - 1 not in s:
            length = 1
            while num + length in s:
                length += 1
            longest = max(longest, length)

    return longest

# Example: [100,4,200,1,3,2] -> 4
```

Complexity: Time: $O(n)$ average | Space: $O(n)$

3. 3Sum

Explanation: Idea: Sort the array first. Fix one number and use two pointers for the remaining two numbers. Skip duplicate values so that duplicate triplets are not produced.

Python Solution:

```
def threeSum(nums):
    nums.sort()
```

```

result = []

for i in range(len(nums) - 2):
    if i > 0 and nums[i] == nums[i - 1]:
        continue

    left, right = i + 1, len(nums) - 1

    while left < right:
        total = nums[i] + nums[left] + nums[right]

        if total == 0:
            result.append([nums[i], nums[left], nums[right]])
            left += 1
            right -= 1

            while left < right and nums[left] == nums[left - 1]:
                left += 1
            while left < right and nums[right] == nums[right + 1]:
                right -= 1

        elif total < 0:
            left += 1
        else:
            right -= 1

    return result

# [-1,0,1,2,-1,-4] ->
# [[-1,-1,2],[-1,0,1]]

```

Complexity: Time: $O(n^2)$ | Space: $O(1)$ auxiliary, ignoring sorting/output

4. Daily Temperatures

Explanation: Idea: Use a monotonic decreasing stack containing indices. When the current temperature is warmer than the temperature at the stack's top index, we have found the answer for that earlier day.

Python Solution:

```

def dailyTemperatures(temperatures):
    ans = [0] * len(temperatures)
    stack = [] # indices

    for i, temp in enumerate(temperatures):
        while stack and temp > temperatures[stack[-1]]:
            j = stack.pop()
            ans[j] = i - j

        stack.append(i)

    return ans

# [73,74,75,71,69,72,76,73]
# -> [1,1,4,2,1,1,0,0]

```

Complexity: Time: $O(n)$ | Space: $O(n)$

5. Kth Largest Element in an Array

Explanation: Idea: Use a min-heap of size k. Keep only the k largest values seen so far. The smallest value in this heap is therefore the kth largest value overall.

Python Solution:

```

import heapq

def findKthLargest(nums, k):
    heap = []

    for num in nums:
        heapq.heappush(heap, num)

        if len(heap) > k:
            heapq.heappop(heap)

    return heap[0]

```

```
    return heap[0]

# nums = [3,2,1,5,6,4], k = 2
# answer = 5
```

Complexity: Time: $O(n \log k)$ | Space: $O(k)$

6. Merge K Sorted Linked Lists

Explanation: Idea: Put the first node from every non-empty linked list into a min-heap. Repeatedly remove the smallest node, attach it to the result, and add that node's next node to the heap.

Python Solution:

```
import heapq

def mergeKLists(lists):
    heap = []
    counter = 0

    for node in lists:
        if node:
            heapq.heappush(heap, (node.val, counter, node))
            counter += 1

    dummy = ListNode(0)
    tail = dummy

    while heap:
        value, _, node = heapq.heappop(heap)
        tail.next = node
        tail = node

        if node.next:
            heapq.heappush(
                heap, (node.next.val, counter, node.next)
            )
            counter += 1

    return dummy.next
```

Complexity: Time: $O(N \log k)$, where N is the total number of nodes | Space: $O(k)$

7. Course Schedule II

Explanation: Idea: This is a topological sorting problem. Build a directed graph and indegree array. Start with courses having indegree 0. Remove them and reduce the indegree of their neighbors. If all courses are processed, the ordering is valid; otherwise there is a cycle.

Python Solution:

```
from collections import deque

def findOrder(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses

    for a, b in prerequisites:
        graph[b].append(a)
        indegree[a] += 1

    q = deque(i for i in range(numCourses)
              if indegree[i] == 0)

    order = []

    while q:
        course = q.popleft()
        order.append(course)

        for nxt in graph[course]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                q.append(nxt)

    return order if len(order) == numCourses else []

# Example -> [0,2,1,3] is one valid answer
```

Complexity: Time: $O(V + E)$ | Space: $O(V + E)$

8. Lowest Common Ancestor of a Binary Tree

Explanation: Idea: Recursively search the left and right subtrees. If the current node is p or q, return it. If both sides return a node, the current node is the lowest common ancestor. Otherwise return the non-empty side.

Python Solution:

```
def lowestCommonAncestor(root, p, q):
    if root is None or root == p or root == q:
        return root

    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)

    if left and right:
        return root

    return left if left else right

# For p = 5 and q = 1 in the given tree,
# the answer is node 3.
```

Complexity: Time: $O(n)$ | Space: $O(h)$ recursion stack, where h is tree height

9. Word Search

Explanation: Idea: Use DFS/backtracking from every cell that matches the first character. Mark a cell as visited during the current path, explore four directions, then restore the cell when backtracking.

Python Solution:

```
def exist(board, word):
    rows, cols = len(board), len(board[0])

    def dfs(r, c, i):
        if i == len(word):
            return True

        if (r < 0 or r >= rows or c < 0 or c >= cols
            or board[r][c] != word[i]):
            return False

        temp = board[r][c]
        board[r][c] = '#'

        found = (
            dfs(r + 1, c, i + 1) or
            dfs(r - 1, c, i + 1) or
            dfs(r, c + 1, i + 1) or
            dfs(r, c - 1, i + 1)
        )

        board[r][c] = temp
        return found

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True

    return False

# "ABCCED" -> True
```

Complexity: Time: $O(m \cdot n \cdot 4^L)$ worst case | Space: $O(L)$, where L is word length

10. Longest Increasing Subsequence

Explanation: Idea: Maintain a list called tails. tails[i] stores the smallest possible ending value of an increasing subsequence of length i+1. Use binary search to find where each number belongs.

Python Solution:

```
from bisect import bisect_left

def lengthOfLIS(nums):
    tails = []

    for num in nums:
        pos = bisect_left(tails, num)

        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num

    return len(tails)

# [10,9,2,5,3,7,101,18] -> 4
# One LIS is [2,3,7,101]
```

Complexity: Time: $O(n \log n)$ | Space: $O(n)$